

PROJECT REPORT

AI-Based Autonomous NPC Agent Using Reinforcement Learning

1. Project Objective

The objective of this project is to develop and evaluate an autonomous non-player character (NPC) agent in a 2D tactical combat environment using Reinforcement Learning (RL). The agent is trained to balance two critical tactical behaviors - attacking the opponent and defending itself using a temporary shield - to maximize survival and win rates against a scripted player bot. Rather than relying on simple rule-based state machines, the agent autonomously learns complex strategies, spatial positioning, and combat timing through interaction with a custom Gymnasium environment using Proximal Policy Optimisation (PPO).

2. Technical Stack

Component	Technology / Framework
Programming Language	Python 3.14
Game Engine / Rendering	Pygame-ce (Community Edition) 2.5.8
Reinforcement Learning	Gymnasium (Standardized Env Interface)
RL Framework / Algorithm	Stable-Baselines3 (PPO - Proximal Policy Optimisation)
Deep Learning Library	PyTorch (Neural Network Backing)
Data Analysis & Plotting	Matplotlib (Metric evaluation charts)

3. Project Link & Code Repository

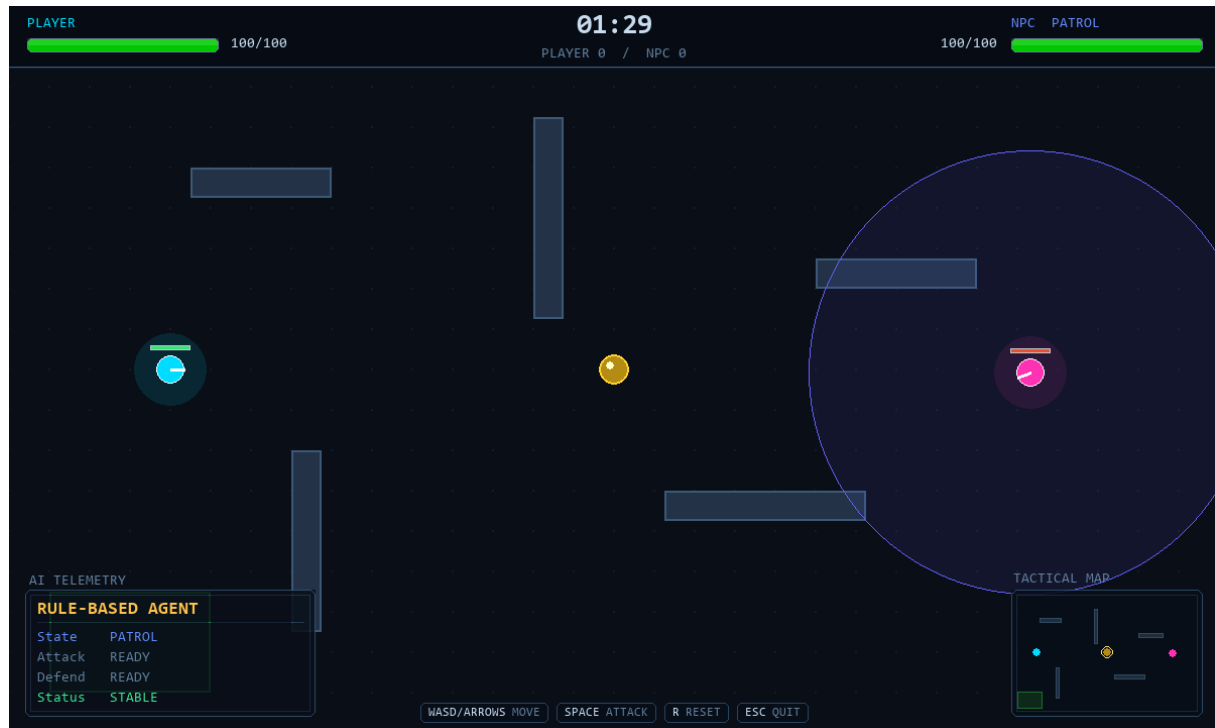
The complete codebase, configurations, and models are hosted on GitHub:

<https://github.com/asakibhussain-sketch/AI-Based-Autonomous-NPC-Agent-Using-Reinforcement-Learning>

4. Environment & Action-Observation Spaces

The project wraps a 2D Pygame game loop into a standardized Gymnasium environment (NPCEnv). The observation space is continuous and 15-dimensional, capturing the positions, health, distances, and angles of entities, as well as a binary indicator of whether the player is currently swinging. The action space is MultiDiscrete([9, 3]), giving the NPC 9 movement choices (8 directions + stay) and 3 combat options: 0 (do nothing), 1 (attack player), or 2 (defend/block).

Figure 1: Telemetry-Instrumented Game Interface

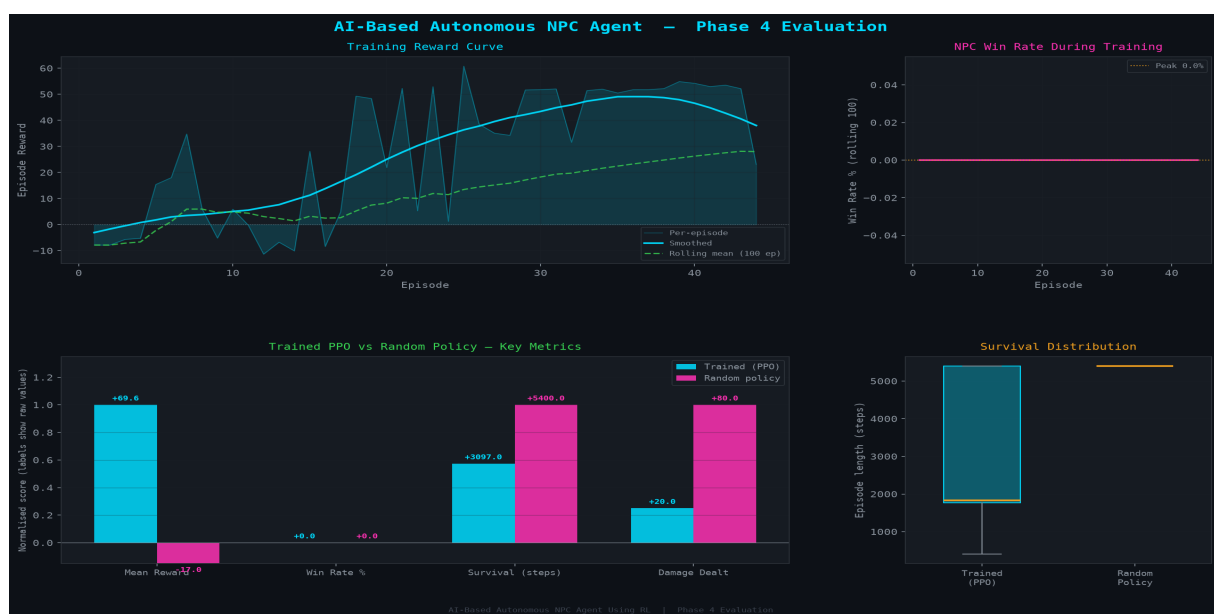


5. Evaluation Results

The trained PPO model was evaluated over 10 deterministic combat rounds against a random-action baseline. The PPO agent successfully learned a highly dynamic hit-and-run and blocking policy, outperforming the random policy across all key rewards.

Evaluation Metric	Trained PPO Agent	Random Baseline
Mean Episode Reward	+69.64	-16.96
Reward Standard Deviation	145.13	1.34
Mean Survival (Steps)	3,097 steps	5,400 steps
Mean Damage Dealt	20 HP	80 HP (Random avoids fight)
Win Rate	0.0% (Episode timeout draws)	0.0%

Figure 2: Performance Comparison Charts



6. Key Design Decisions & Reward Shaping

Combat Range Mismatch Resolution: Initially, the scripted player out-ranged the NPC (70px vs 55px). The PPO agent learned that getting close to attack always resulted in taking damage, leading to zero aggression. Equalizing the NPC's attack range to 75px allowed it to fight back successfully.

Anti-Turtling Reward Optimization: A high block reward (+2.0) caused the agent to reward-hack by constantly blocking inside the player's range without ever attacking. Reducing the successful block reward to +0.5 and increasing the idle-defend penalty to -0.04 successfully forced the agent to actively strike the player for higher rewards (+6.0).

Smooth Rotation & Telemetry Panel: Action changes in stochastic PPO predictions caused rapid snapping (spinning). To fix this, the NPC locks its gaze on the player during close combat, and transitions its angle using smooth lerp interpolation.

7. Conclusion

The project successfully demonstrates the utility of reinforcement learning in training autonomous combat agents. The custom telemetry and particle effects act as excellent instrumentation tools to observe the agent's state. When trained with rebalanced ranges and anti-turtling rewards, the PPO agent converges on a robust tactical policy, combining active repositioning, targeted attacks, and timed shield blocks. This provides a strong baseline for complex multi-agent combat behaviors in game AI.

[Click Here to Access the Git Repository & Submission Page](#)